QA Engineer Home Assignment - Scoped

- Overview
- Required Stack
- Application Access
- Part A - Manual QA & Test Strategy
  - 1) Test Plan
  - 2) Execute Your Top 3 Scenarios
- Part B - Automation
  - 3) Framework + 2 Automated Tests
- Part C - Strategy & Recommendations
- Submission Checklist
- Domain Context
  - Betting Concepts

Overview

This assignment evaluates how you move from understanding a feature, to identifying risk, to building targeted automation. You will work with a web application that simulates a sports betting flow.

We value strategic thinking and clean engineering over exhaustive coverage. A focused submission with clear rationale will score higher than a large one with weak justification.

Required Stack

- **Language:** Python 3
- **UI automation:** Selenium WebDriver + Pytest
- **API testing:** Python requests library
- **Browser:** Latest desktop Chrome

If you use additional tooling (Poetry, Allure, etc.), document your choices briefly in the README.

Application Access

Test application: https://qae-assignment-tau.vercel.app/

Authentication: append your user-id as a query parameter:

`https://qae-assignment-tau.vercel.app/?user-id=<your-user-id>`

API docs are available at /api/docs (Swagger UI).

Part A - Manual QA & Test Strategy

1) Test Plan

Review the **Single Bet Placement Feature Specification document** and design 5-6 prioritized test scenarios. Your set should demonstrate range across happy paths, negative/validation cases, and boundary conditions. For each scenario, use:

- **ID and Title**
- **Priority**: Critical / High / Medium / Low
- **Risk Rationale**: Use it to show you understand which risks drive your test selection.
- **Steps**
- **Expected Result**

Create this test plan in an individual md file

2) Execute Your Top 3 Scenarios

Run your 3 highest priority scenarios against the application. Also spend a few minutes on quick exploratory checks around the bet placement flow

For each defect you find during execution or exploration, report it using:

- **Bug ID & Title**
- **Severity:** Critical / High / Medium / Low
- **Reproduction Steps**
- **Expected vs Actual result**
- **Business Impact:** Brief sentence on user/business consequence
- **Evidence:** Screenshot or brief note

Document this in an individual md file

You are not scored on bug count. You are scored on the quality of your reports and whether you catch the high impact issues

Part B - Automation

3) Framework + 2 Automated Tests

Build a small, well structured automation project and implement 2 high value automated tests:

- **E2E UI test:** a critical user journey
- **API test:** a validation or business rule check via the API directly

For each test, include a brief comment or docstring explaining why you chose it.

We evaluate your framework more than your tests. Among other things, we are looking for:

- Clean project structure
- Readable, maintainable code following Python conventions
- A README with clear setup and run instructions

Part C - Strategy & Recommendations

Write a short section covering:

- Why you selected these 2 tests for automation over other candidates
- What you intentionally left as manual only and why
- Your top 2–3 recommendations if this project were to scale (CI/CD, additional test layers, data strategy, spec clarifications, etc)

Document this in an individual md file

Submission Checklist

All deliverables must be included in a Github public repository

- Test plan (5-6 prioritized scenarios)
- Execution results / bug reports
- Automation framework and 2 tests
- Strategy and recommendations note

Domain Context

Betting Concepts

- **Match / Event** — A scheduled football game between two teams (e.g., Manchester Utd vs Chelsea). Each match has a competition (league), kickoff date/time, and three possible outcomes.
- **Odds** — Decimal numbers representing the payout multiplier for each outcome. Displayed as three buttons per match: **1** (home win), **X** (draw), **2** (away win). Higher odds = less likely outcome = bigger payout. For example, odds of 2.45 mean a €10 bet returns €24.50.
- **Stake** — The amount of money the user risks on a bet. Entered in euros (€). Must be a positive number and cannot exceed the user's current balance.
- **Payout** — The total amount returned to the user if the bet wins. Calculated as **stake × odds**. This includes the original stake — so a €10 bet at 2.45 odds returns €24.50 total (€14.50 profit + €10 stake).
- **Balance** — The user's available funds. Starts at €125.50. Decreases by the stake amount when a bet is placed. The balance is shared across the header and the bet slip.
- **Bet Slip** — The right-hand panel where the user reviews their selection, enters a stake, and places the bet. Only one bet can be active at a time.
- **Bet Receipt** — The success modal shown after a bet is placed. Displays the bet ID, match, stake, odds, potential payout, and timestamp. All values should be consistent with what was shown before placement.
- **Match Ordering** — Matches are displayed as returned by the API. The "home" team is always listed first (left position), the "away" team second (right position). This convention carries through to the bet receipt (e.g., "Manchester Utd vs Chelsea" means Manchester Utd is home).